TaskFlow Solutions

Todo List Full-Stack Application

Business Confidential

Date: 18th of November, 2026
Author: 5724102
Version: 1.0

Contents

1 Secure Design **3**
- 1.1 Application Architecture . . . 3
  - 1.1.1 Overall Structure . . . 3
  - 1.1.2 How layers interact . . . 3
- 1.2 Class Diagram . . . 4
  - 1.2.1 UML . . . 4
  - 1.2.2 DFD and STRIDE . . . 6
  - 1.2.3 Important classes . . . 7
- 1.3 Secure Design Principles . . . 7
  - 1.3.1 Least Privilege . . . 7
  - 1.3.2 Separation of concerns . . . 7
  - 1.3.3 Input validation . . . 7
  - 1.3.4 Defence in depth . . . 7

2 Implementation **8**
- 2.1 Primary Requirements . . . 8
  - 2.1.1 Task Status . . . 8
  - 2.1.2 Task Priority . . . 8
  - 2.1.3 Task Descriptions . . . 8
- 2.2 Organizational Features . . . 8
  - 2.2.1 Task Tags . . . 8
  - 2.2.2 Dashboard . . . 8
- 2.3 Authentication and Security . . . 8
  - 2.3.1 User Login . . . 8
  - 2.3.2 Secure Password Handling . . . 8
  - 2.3.3 Session and Token Management . . . 9
  - 2.3.4 Enumeration Prevention . . . 9
  - 2.3.5 HTTPS . . . 9
- 2.4 Implementation Quality . . . 9
  - 2.4.1 Input Validation and Error Handling . . . 9
  - 2.4.2 Database Management . . . 9
  - 2.4.3 Code Organisation and Maintainability . . . 9

3 Evidence **10**
- 3.1 Primary Requirements . . . 10
  - 3.1.1 Task Status . . . 11
  - 3.1.2 Task Priority . . . 11
  - 3.1.3 Task Descriptions . . . 11
- 3.2 Organizational Features . . . 12
  - 3.2.1 Task Tags . . . 12
  - 3.2.2 Dashboard . . . 12
- 3.3 Authentication and Security . . . 13
  - 3.3.1 User Login . . . 13
  - 3.3.2 Secure Password Handling . . . 13
  - 3.3.3 Session and Token Management . . . 14
  - 3.3.4 Enumeration Prevention . . . 15

4 Critical Reflection **16**
4.1 Agile Planning 16
4.2 Comparison of Security Approaches 16
4.3 Secure Development Across Lifecycle 16
4.4 Limitations and Future Development 17

Appendices **18**

A GitLab Commit History **18**

B Repository **21**

C AI Statement **21**

1 Secure Design

1.1 Application Architecture

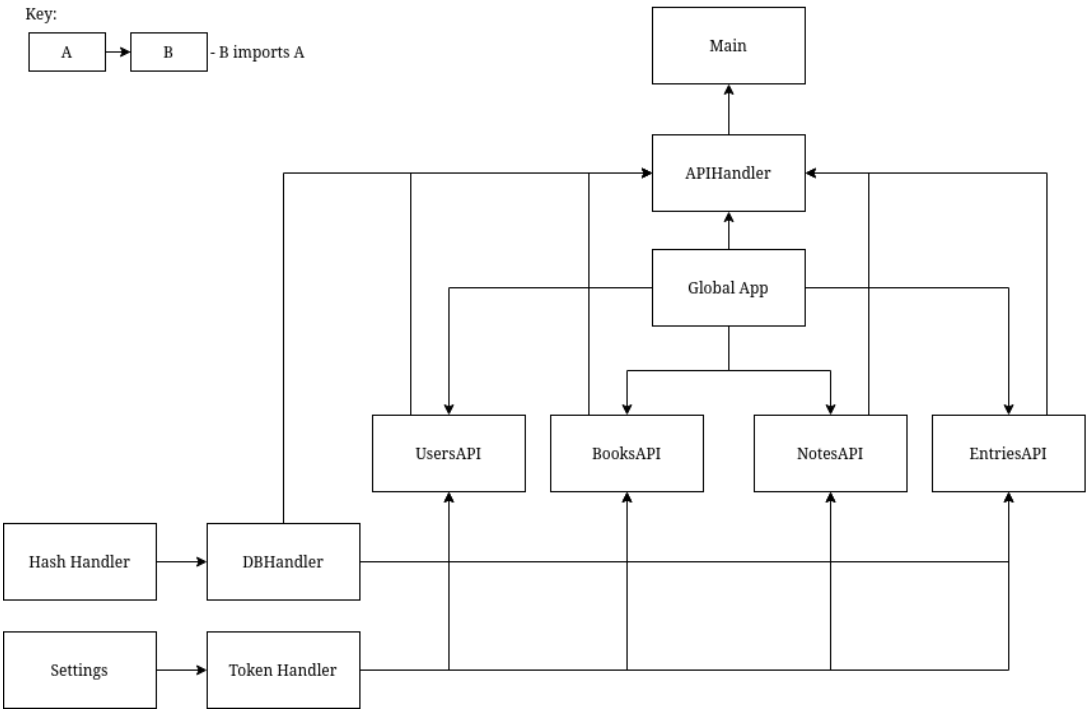


Figure 1: Python Import Structure

1.1.1 Overall Structure

The application separates responsibilities into the frontend, backend, and the database. The frontend manages user interaction and preliminary validation, the backend enforces authorization, authentication, and input validation, and the database manages storage. This separation improves maintainability and ensures security is enforced server-side, preventing client-bypass vulnerabilities.

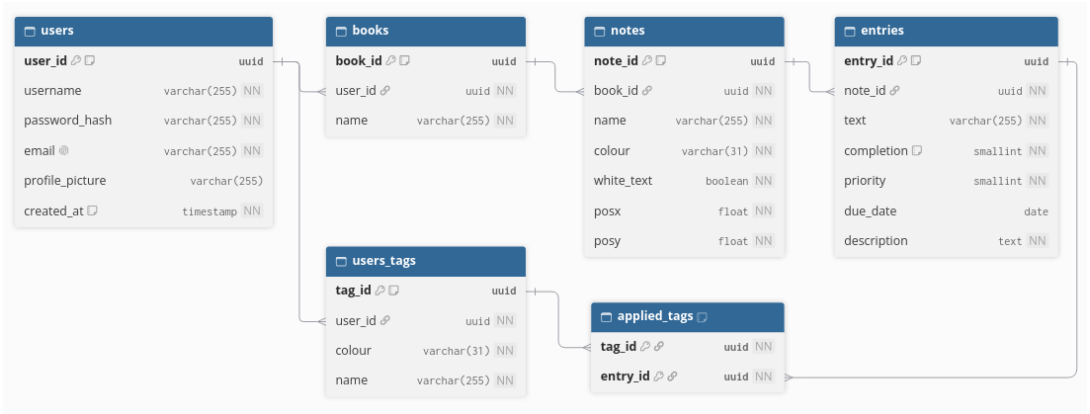


Figure 2: PostgreSQL Database Design.

1.1.2 How layers interact

The frontend handles and validates user interaction before sending it to the REST API in the backend, which validates the JWT, ensuring the user is authenticated and authorised. It also validates the request,

and sends it to the data management layer, where required queries are performed. Data then travels through the backend and back to the frontend, where it is displayed for the user.

1.2 Class Diagram

1.2.1 UML

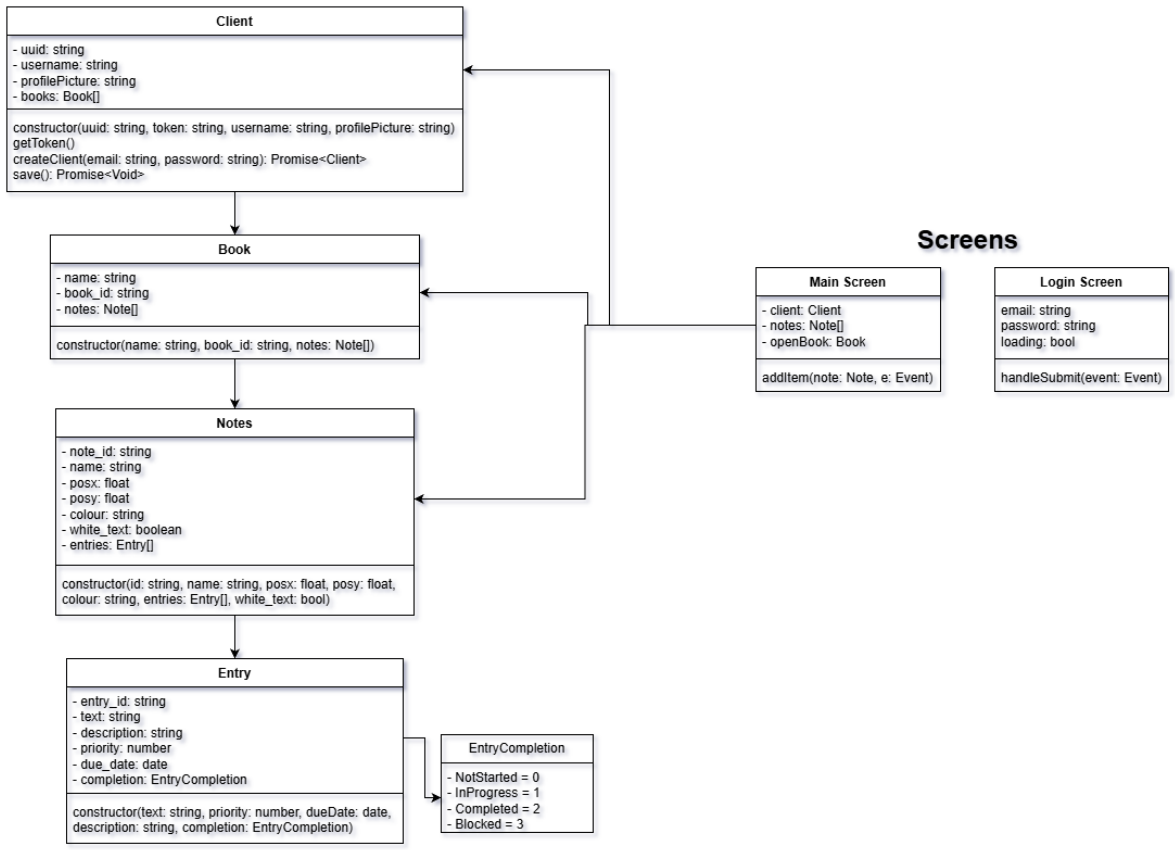


Figure 3: Frontend UML Diagram

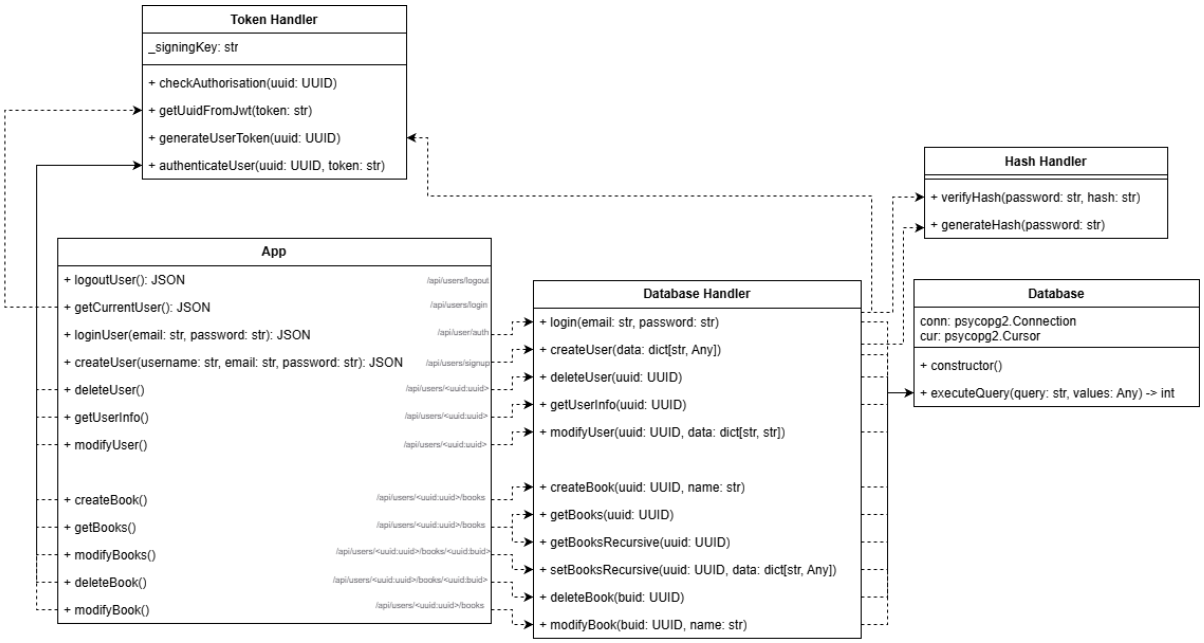


Figure 4: Backend UML Diagram

1.2.2 DFD and STRIDE

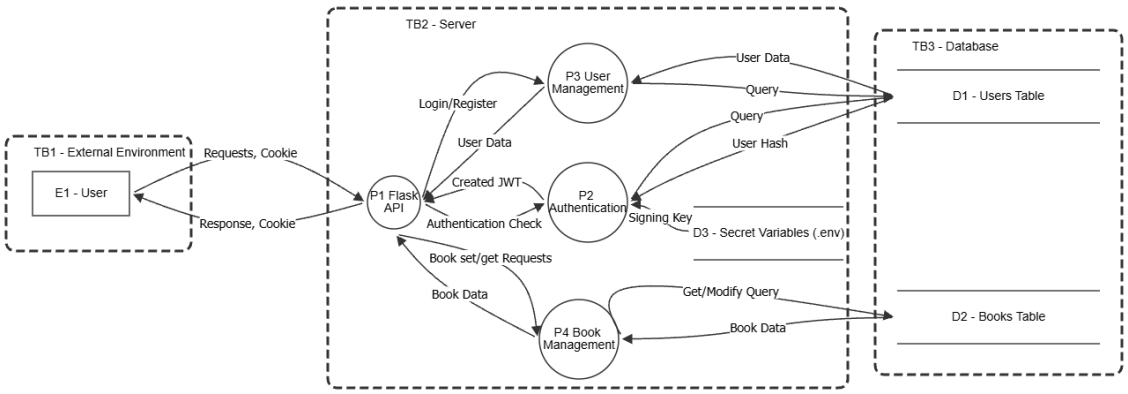


Figure 5: DFD Diagram

ID	Name	Description
TB1	External Environment	The internet
TB2	Server	The server running the service
TB3	Database	The PostgreSQL database.
E1	User	User Client.
P1	Flask API	API handler.
P2	Authentication	Handles user authentication and token management.
P3	User Management	Handles Users.
P4	Book Management	Handles Books.
D1	Users Table	Contains user information.
D2	Books Table	Contains books.
D3	Secret Variables (.env)	Contains environment variables.

Table 1: DFD Elements

STRIDE Category	Affected ID	Threat	Mitigation
Spoofing	E1, P2	Impersonation of a legitimate user via stolen or forged credentials	Ensure cookies are HTTP-Only, ensure JWTs are valid, ensure JWTs expire
Tampering	P1	Modification of requests to manipulate application behaviour	Validate users are authorised to access material.
Tampering	E1	Bypassing client side validation	Ensure authorisation occurs server side.
Tampering	P3, D2	Modification of user records to gain authorisation	Use minimal database permissions, use server side authorisation
Tampering	P2	Tampering with JWT contents to alter claims	Verify JWT signatures using secure algorithm

STRIDE Category	Affected IDs	Threat	Mitigation
Information Disclosure	P1	Internal errors are exposed through API request	Return generic error messages.
Information Disclosure	P2	JWTs or credentials are exposed through insecure communication	Ensure HTTPS is enabled, use secure and HTTPOnly cookies.
Information Disclosure	D3	Environment variables accessed, containing secret information like signing keys	Exclude .env from git, use a secrets manager

Table 2: STRIDE Threat Assessment

1.2.3 Important classes

In the frontend, Client stores user information such as books, tags, and their UUID. This also provides an abstraction layer between the TSX components and the API.

Frontend display is divided into two screens: Main and Login. Main handles app interactions and variables such as open book and notes. Login handles logging in and signing up, as well as Client creation.

The backend uses the Flask 'app' class to manage the API, and the database is handled by a 'database' class, which acts as an abstraction layer between the backend and the PostgreSQL database.

1.3 Secure Design Principles

1.3.1 Least Privilege

Least privilege ensures users have minimal privilege required to complete tasks, minimising attack surface, for example users only being able to access their own notes.

1.3.2 Separation of concerns

Separation of concerns divides complex problems into smaller, independent components with distinct responsibilities. This improves modularity and readability.

The application implements this in three layers: frontend, backend, and data management. With each having a separate purpose, changes cause minimal changes to other layers.

1.3.3 Input validation

Input validation occurs primarily in the backend, as frontend validation can be bypassed. However, frontend validation still occurs to reduce unnecessary server load.

Validation ensures data is correct and prevents malicious payloads, reducing the risk of injection vulnerabilities.

1.3.4 Defence in depth

Defence in depth uses multiple layers of security to reduce chances of a security breach, as bypassing one layer does not compromise the entire application.

This is implemented through backend authentication, authorisation, input validation rejecting invalid requests, and parametrised SQL queries preventing SQL injection. These measures prevent unauthorised access through many layers of security.

2 Implementation

2.1 Primary Requirements

2.1.1 Task Status

Task status' represent the progress of a given task, with values consisting of *Not Started*, *In Progress*, *Completed*, or *Blocked*. These are implemented using an Enum, which will contain the set of possible values, improving consistency and readability in the codebase.

In the database, the Enum is stored as a `smallint`, this more compact integer allows for expansion of the possible values while keeping memory costs low.

The user modifies a tasks status by changing it in a drop-down menu as there is a set list of possible values. The status is shown as a coloured circle with a tooltip of the status name so that users can quickly see a tasks current status.

2.1.2 Task Priority

Task priority represent how important a given task is, with values consisting of *Low*, *Medium*, *High*, or *Critical*. These are implemented using an Enum, which contains a set of all possible values, for the same reasons as status.

In the database, this will also be saved as `smallint`s, which again allow for expansion while reducing storage load.

The user can select priority by either setting it in task creation, or by changing it while editing the task.

2.1.3 Task Descriptions

Task descriptions are stored as a PostgreSQL `text` field to be able to handle variable length user input. Users can create and edit descriptions.

2.2 Organizational Features

2.2.1 Task Tags

Task tags allow the user to give different entries categories, which can be used to quickly see which tasks are of a certain category when paired with searching.

These tags are stored in a table, with a `varchar(255)` for the name, and a `varchar(31)` for the colour.

The tags can be created and applied when the tag is being edited.

2.2.2 Dashboard

The dashboard allows the user to see how many entries there are, broken down by priority, completion, or by tag. This is done per book.

2.3 Authentication and Security

2.3.1 User Login

Users are able to login to the system by sending their email and password to the server, where it checks that the password matches and on success returns a cookie containing the JWT that allows the user to access their notes.

2.3.2 Secure Password Handling

Passwords are stored with Argon2 hashes instead of plaintext to prevent password leaks in a data breach. Argon2 is a slow hashing algorithm, making brute force attacks computationally expensive.

2.3.3 Session and Token Management

Upon login, users receive a cookie containing a JWT with their UUID and expiry date. When accessing data the server validates the token to verify authentication, authorisation, and no expiry date.

The cookie is HTTP only, preventing JavaScript access and reducing risk of token theft, however this does not stop all cross site attacks. CSRF protection is implemented via a CSRF token, validated by Flask-WTF.

Tokens expire after 24 hours, a duration configurable in the settings file. This limits an attackers duration in an account after stealing their token.

2.3.4 Enumeration Prevention

To prevent username enumeration, the server returns an error 401 on unauthorized access and on attempt to access a user that does not exist, as if a 404 was returned the user could know if that user exists.

2.3.5 HTTPS

HTTPS is currently disabled while on a testing server, however once this server is put into production HTTPS must be enabled to ensure no man in the middle attacks occur on the users passwords.

2.4 Implementation Quality

2.4.1 Input Validation and Error Handling

Inputs are validated across the frontend, backend, and data management layers.

The database handler wraps queries in try/except blocks, returning only safe, predictable responses to the backend. This prevents revealing information about the application through internal database errors.

2.4.2 Database Management

The database is managed by "*DatabaseHandler.py*", which acts as an abstraction layer between the backend and PostgreSQL, improving backend readability and allowing database changes with minimal changes to other sections of code.

2.4.3 Code Organisation and Maintainability

The code for this application has been properly separated into individual sections, including the API, which has been split up into each type of endpoint, and the database, which has its own handler.

This will improve modularity and improve maintainability as the code is more readable.

3 Evidence

3.1 Primary Requirements

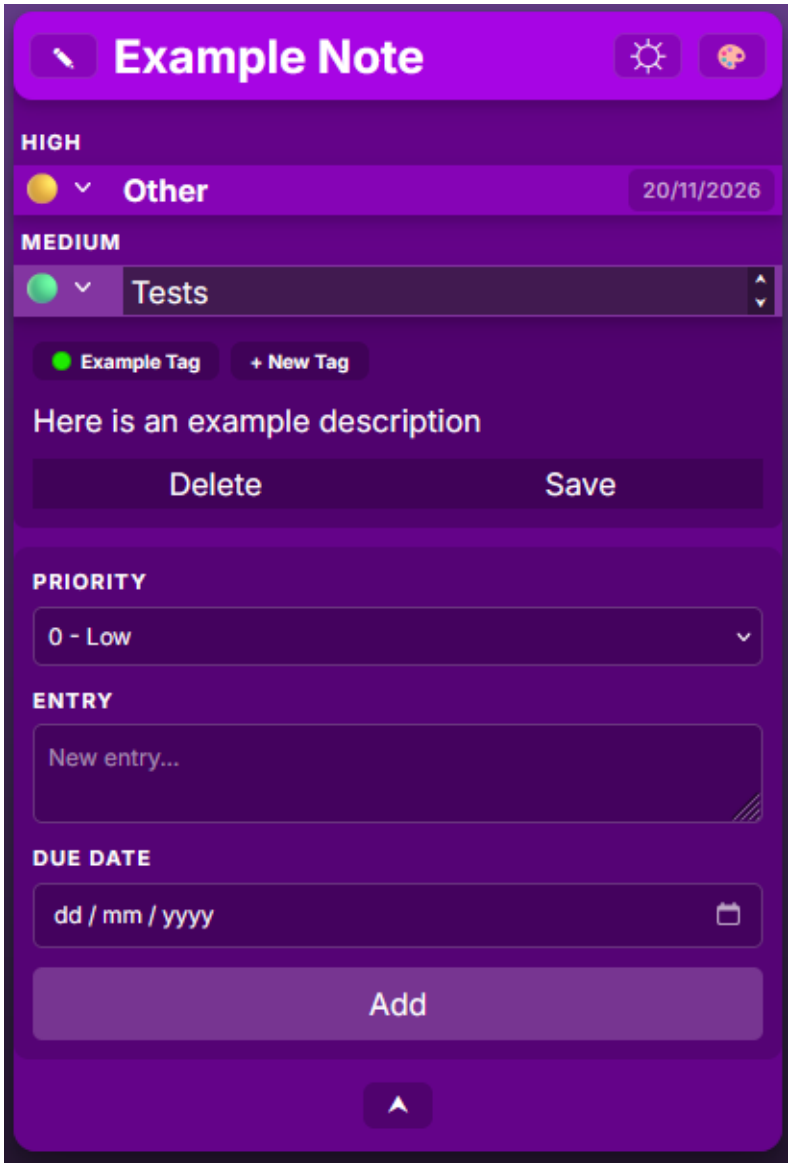


Figure 6: Note Example

3.1.1 Task Status

Four tasks were assigned each of the statuses: 'Not Started', 'In Progress', 'Completed', and 'Blocked', demonstrated in Figure 7, which confirmed that all states are functional, as well as the selector.

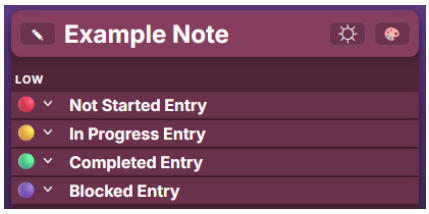


Figure 7: Status Example

3.1.2 Task Priority

Eight tasks were created with varying priority levels. Another task was created and moved between priorities to verify priority modification was functional. (Figure 8)

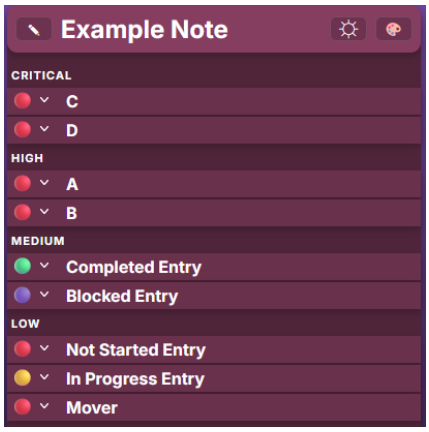


Figure 8: Priority Example

3.1.3 Task Descriptions

Task descriptions can be edited by opening an entry and clicking on the description, as seen in Figure 6.

To verify functionality, several entries descriptions were edited, and each was checked to ensure description remained after clicking off the entry, as seen in Figure 9

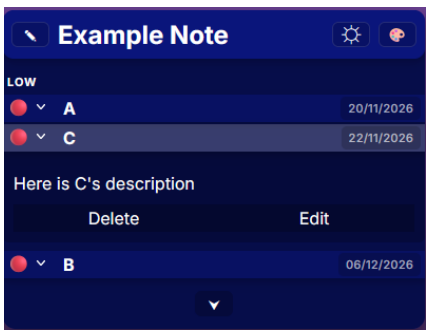


Figure 9: Description Example

3.2 Organizational Features

3.2.1 Task Tags

Task tags can be added by editing an entry and pressing add tag, or removed by clicking on the tag. The colour can be changed by clicking on the coloured circle. Creating or deleting tags occurs in the add tag menu, with an X to delete, and the '*+ Create new tag*' button on the bottom to create, as seen in Figure 6.

To verify functionality, several tags were created and applied to different entries, as seen in Figure 10.

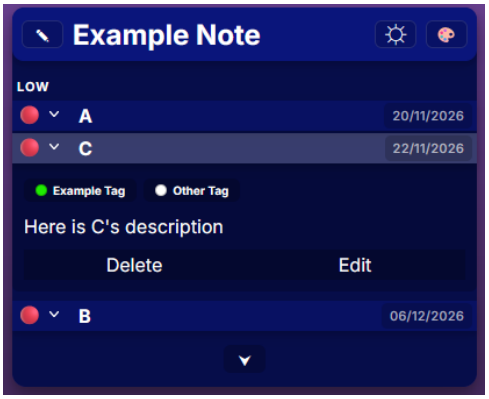


Figure 10: Tag Example

3.2.2 Dashboard

The dashboard allows users to see how many entries are in the current book, filtered by status, priority, or by tag, as seen in Figure 11.

To verify functionality, several entries were changed while the dashboard was open, and counts were checked to be correct.

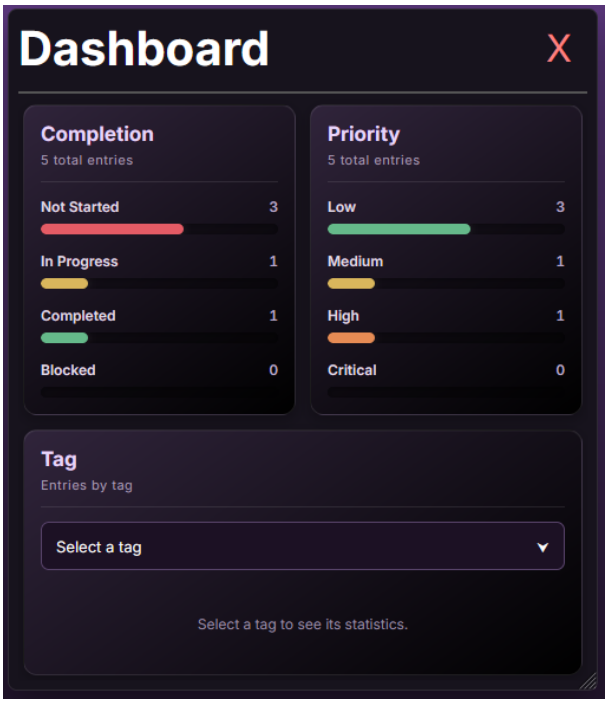


Figure 11: Dashboard Evidence

3.3 Authentication and Security

3.3.1 User Login

Users can log in through a simple UI. Functionality was tested with multiple accounts to verify successful authentication and correct loading of books and notes, as seen in Figure 12 and Figure 13.

Invalid credentials are rejected, requiring user to reattempt. Return messages do not reveal whether an account exists, preventing username enumeration.

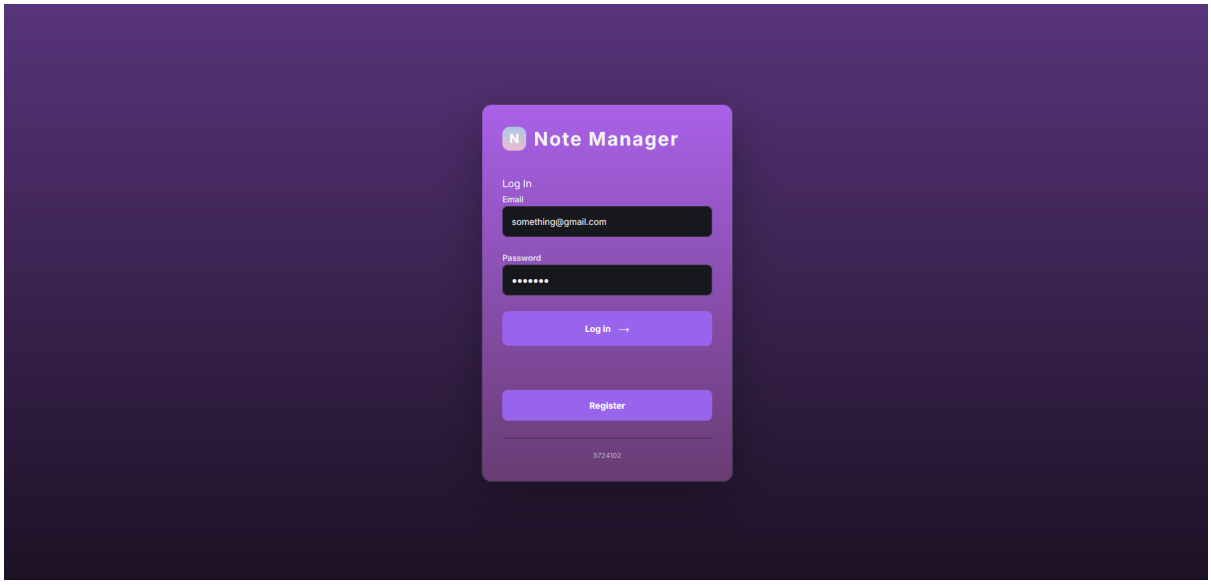


Figure 12: Login Screen

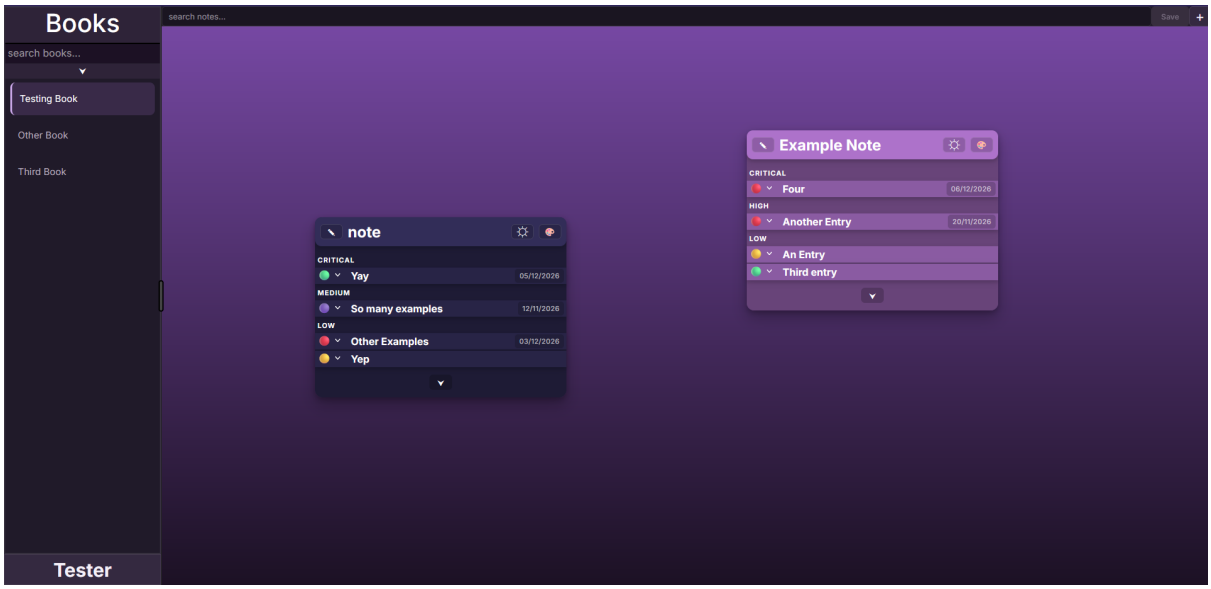


Figure 13: Successful login and loading

3.3.2 Secure Password Handling

While not currently enabled, when in production HTTPS will be used to transmit the users plaintext password to the server.

When the password reaches the server, it verifies it against the Argon2 hashed password in the database, if valid then it allows login, else it denies. The plaintext password is never stored, as seen in

Figure 14 and Figure 15.

```python
@app.route('/api/users/login', methods=['POST'])
def loginUser():

    data = request.get_json()
    email = data["email"]
    password = data["password"]
    if email == None or password == None:
        return {"error": "missing email or password in parameters"}

    result = DBHandler.login(email, password)

    if "error" in result:
        return result, 400

    token = result.get("token")
    if token == None:
        return {"error": "login did not return token"}, 500

    response = jsonify({
        "uuid": result.get("uuid")
    })

    response.set_cookie(
        key="access_token",
        value=token,
        httponly=True,
        secure=False,
        samesite="Lax",
        path="/",
        max_age=Settings.tokenDuration * 60 * 60
    )

    return response
```

Figure 14: Login API

```python
def login(email: str, password: str) -> dict[str, str]:

    database.executeQuery("""
    SELECT password_hash, user_id FROM public.users
    WHERE email = %s
    """, (email,))

    result = database.cur.fetchone()

    if result == None:
        return {"error": "Password was incorrect or user does not exist"}

    if HashHandler.verifyHash(password, result[0]) == True:
        token = TokenHandler.generateUserToken(result[1])
        return {"token": token, "uuid": result[1]}

    return {"error": "Password was incorrect or user does not exist"}
```

Figure 15: Login DB

3.3.3 Session and Token Management

Sessions are continued via a HTTP only cookie that contains a JWT that authenticates the user. The user receives this cookie on login, as seen in Figure 14. This JWT is handled by a JWT handler class, improving modularity, as seen in Figure 16.

```python
def generateUserToken(uuid: UUID):
    now = datetime.datetime.now()

    claims = {
        "uuid": uuid,
        "createdAt": str(now.astimezone()),
        "expiryDate": str((now + datetime.timedelta(hours=Settings.tokenDuration)).astimezone())
        }

    token = jwt.encode(
        headers={"typ": "JWT"},
        key=_signingKey,
        payload=claims,
        algorithm="HS256"
        )

    return token


def authenticateUser(uuid: UUID, token: str) -> bool:
    """
    True if that user is who they claim to be and the token is not expired
    """

    decodedToken: dict[str, Any]

    try:
        decodedToken = jwt.decode(token, _signingKey, algorithms="HS256")
    except jwt.exceptions.InvalidSignatureError:
        print("Invalid signature detected")
        return False # not valid

    expiryDate: datetime.datetime = datetime.datetime.fromisoformat(decodedToken["expiryDate"])
    if expiryDate < datetime.datetime.now().astimezone():
        return False

    return str(uuid) == decodedToken["uuid"]
```

Figure 16: Token Handler Functions

3.3.4 Enumeration Prevention

API responses could allow for username enumeration if different status codes were returned for existing users and those that don't (401 vs 404).

4 Critical Reflection

4.1 Agile Planning

This application was developed using incremental development, allowing for features to be focused on individually, leading to a more modular structure. Each feature was initially a git issue, it was planned out, developed, and tested. This is seen in several commits such as:

- `23ad6bfd` - 'Added REST API plan'
- `7a042224` - 'Added database design'
- `4b652dfa` - 'Added login screen functionality'
- `828cc627` - 'Added searching for tags and description'
- `9e4d6b0c` - 'Fixed priority moving bug'

The implemented features were chosen to improve user experience, with several organizational features. Other features were chosen to improve security, such as secure password handling and a secure login system.

However, incremental development can cause a lack of a holistic view, leading to poor integration between components.

4.2 Comparison of Security Approaches

During development there were two approaches to validation: Frontend vs Backend. Frontend validation reduces server load, as invalid requests are never sent, it also gives immediate feedback. However, frontend validation can be bypassed so it is not a trusted security boundary. Backend validation requires server side processing, however it can't be avoided.

Backend validation was selected as the primary validation system, with frontend validation providing additional non-security benefits.

Another decision required during development was JWT authentication vs server side sessions.

JWT authentication allows for stateless authentication. However, token theft needs to be avoided, and revocation is far more difficult than with a server side session.

Sever side sessions require a centralised session handler, session storage, and session management, however session revocation is quick and easy.

JWTs were chosen for this project as they allow for authentication without requiring a query since not database backed checks are required for this application, improving request speed and reducing server load. The 24 hour expiry JWT system trades more expensive revocation for lower server side complexity. However, there are still many benefits of switching to a server side session manager.

4.3 Secure Development Across Lifecycle

Ensuring user data was protected was a key security requirement, as well as maintaining confidentiality, integrity, and preventing unauthorised access. These requirements influenced the design through principles of separation of concerns, authentication, authorisation, and least privilege.

During implementation, Argon2 password hashing was used to protect stored credentials, HTTPOnly JWT cookies were used to stop JavaScript access to authentication tokens, and parametrised queries were used to prevent SQL injection. These measures satisfied the initial security requirements.

Testing included unauthorised access, token expiry, and invalid inputs, However these checks cannot show that this application has no vulnerabilities, so further and more comprehensive security testing will be required.

4.4 Limitations and Future Development

Currently HTTPS is not enabled, which is a serious security vulnerability allowing man in the middle attacks on login, where passwords are transferred in plaintext, as well as the returning authentication cookie. In production HTTPS should be properly deployed and cookies should have required security configurations.

JWTs cannot be immediately revoked, meaning if a users token is stolen their account is compromised for a maximum of 24 hours. Future development could include server side session management. The cookie also currently does not have Secure enabled, which could enable some attacks to be able to obtain the authentication token.

Currently, limited manual security testing has been used, which does not demonstrate resistance against many other possible malicious attacks. Automated tests and more thorough manual tests could improve confidence and find further vulnerabilities, as automated tests could systematically check inputs to the APIs, and more thorough manual tests could find vulnerabilities not covered by ordinary functional testing.

Appendices

A GitLab Commit History

Commit Message	Commit ID
Fixed CSRF token not applying properly	649abb53
Added required secret key to app	6411c098
Merge branch 'main' of https://git-lab.cyber.warwick.ac.uk/shortstuff/ssd-...	4b470bb6
Updated to protect against CSRF	8621dfc8
Removed some old prints	f651d0a4
Moved TokenHandler to Handlers	772c262f
Removed old app.py	35d9b461
Changed priority to use Enum	3d8efeae
Added editing book name	9ff10b4f
Fixed + note alignment issue	78cb5f51
Made save button enable on book creation	0a44d80d
Fixed dashboard not closing bug	52a10b1e
Moved css to individual files and improved design	7e02c6e7
Completed dashboard	f7eff22d
Fixed dashboard not dragging correctly	276560ae
Started dashboard creation	9e668841
Added deleting books	23e40d55
Edited profile display	208dcb8e
Fixed note dragging issue	da651edc
Added registering account	8c4612fa
Fixed notes moving when book bar moves	8cf28a06
Added searching for tags and description	828cc627
Fixed note searching and completed note search styling	07c71896
Added logging out	418a83ae
Added auto-login based on token cookie	e40ca4f2
Started searching system	f71dd67e
Added changing priority	4a67d644
Fixed issue with try	99d7185d
Added try to queries	764e7a47
Fixed name still being editable when other entry selected	cee0f775
Fixed note name wrapping	12619ccb
Made it highlight selected book	f6e6e362
Removed old save button	ceb9af53
Moved Notes into their own file	da9d6a52

Continued on the next page

Commit Message	Commit ID
Added small CLI explainer in README	2bdd0cf4
Fixed issue with tag update query	59c11e01
Added colour selection for tags	6b18d16e
Added deleting tags from user	8761f8e7
Added deleting tags	a67a3c42
Added adding and removing tags	933b4864
Removed old items variable	79efb01e
Removed old getItems function	23e12694
Added users tag loading	555fb75d
Fixed tag saving issue	2515292d
Changed db design so names are not unique for tags per user	56fb5630
Fixed missing brackets cause 'tag.name' instead of tag name	2853b32a
Fixed tiny 1 pixel misalign issue	662c5508
Added tags	7dbfb76f
Updated colour for entry text during editing	3ab91916
Updated database schema	2aff97b9
Added editing names	b111ba24
Fixed name staying after book creation	5e146fab
Removed old copy json button	7078c2ef
Added regex searching	0f6c4c4a
Added searching functionality	4468177d
Added save button disabling if saved	6cd5fc15
Added selecting books	8f0c85dd
Added add note form	fe1e51fe
Fixed issue with database getting deleted	711f10a4
Fixed issue with no books causing errors	9f6038e4
Fixed Run Program issue	070e0224
Fixed issue with date not loading correctly	b2a182ed
Fixed colour issue causing thousands of warnings	cfbdf039
Added add note button	cb57c1f4
Changed db to have varchar 31 instead of varchar 15 for colours	ec62f4a7
Added deleting notes	f224e2e6
Added missing commit and removed unnecessary old ones	a3705b1e
Fixed note uuid type being wrong	5f0abfc4
Fixed tooltip issue	992d7af2
Fixed save button styling	c270f69c
Added deleting entries	ac8dd6c5
Fixed priority moving bug	9e4d6b0c
Completed saving and loading to database	3ffa5e7e

Continued on the next page

Commit Message	Commit ID
Fixed small padding issue with copy button	ba56b24e
Removed side padding from notes	84528de6
Changed main screen background	2ba6442a
Moved main screen css to MainScreen.css	b79f021c
Added loading books from server	8736895a
Changed loginScreen to look better	d9a08cc3
Changed textcolour to match white_text	1446dc81
Changed getBooks to acutally get books	79e2d96e
Reduced size of completion button	e24c166d
Changed database.ini config	d403f3de
Added notes handling for database	6ea8b095
Added tooltip on hover for progress	ebbf1ced
Added getting books and creating books, as well as fixing some typos	abb52b12
Changed which gitignore the database.ini ignore was in	6ace4e1e
Added database.ini to git ignore	f87f2368
Fixed date overflow issue	14971831
Changed status to be selectable to change it	7a2f995a
Updated Database Design	7b1b4d3f
FIxed issue with missing rollback when creating new user that already exists	bd035702
Fixed issue with date checking on token handler	0ae0c3a2
Changed test code	62756aba
Fixed path issue	1b47eeb8
Added expiry date check for tokens	5878984f
Fixed hash issue	84742a72
Added login screen functionality	4b652dfa
Added login screen	582f26e9
Added login	fe1079b9
Added HashHandler	235f66fe
Added password hash to database setup	7cd8259e
Added argon2 to requirements	0ff1cc08
Completed modifyUser function	e219f538
Added getUserInfo	7c2d1a49
Added delete user and create user functionality	4d16a618
Added default values to table and email	56dcb456
Added main testing	2e36da83
Added auto database setup	b324fcdc
added all DBHandler functions	3ac0de2d
Added all API handler files	530aa6af
Completed Users API	eb625370

Continued on the next page

Commit Message	Commit ID
Removed unecessary todo	3570b9fa
Added db Handler	11ffc151
Added settings	f265d471
Updated requirements	c243103d
Added Token Handler	4c11997d
Added REST API plan	23ad6bfd
Added db design	7a042224
modified: frontend/src/Screens/MainScreen.tsx	f54d50dd
Added completion status display	c1de7659
Fixed bug where date was not part of entry button, and updated text colour	c0489d5a
Made ”Click to add description” itallic	28ef1d13
Fixed small positional error	a28a7702
Added editing descriptions	039200ab
Added descriptions	9f8ab9b6
Refactored Code	dbb325ec
Added run program bat file to auto start app	2598984e
Fixed spacing issue	3449a6c9
Added sorting by date as well as level	8efdf2b4
Attempted fix to make mouse pointer instead of grabbing when hovering over note	3a1cc284
Moved critical level to above the entries	a95de0c4
Changed position of edit button	30184da4
Added colour selecting	b32cac6f
Added renaming Notes	19fc8115
Added button to close form for notes and fixed issue with user profile overflowing	0b21abb7
Added form to add entries to notes	1fa52ecf
Added moving notes and listing entries in those notes	1cb501d6
Changed header name and increased resize bar height	553c5671
Added side bar dragging	522ab803
Added detailing on side bar	9196d4c9

Table 3: GitLab Commit Reference

B Repository

To access code, go to ’`https://git-lab.cyber.warwick.ac.uk/shortstuff/ssd-todo-5724102`’ or clone via ’`https://git-lab.cyber.warwick.ac.uk/shortstuff/ssd-todo-5724102.git`’

C AI Statement

No AI tools were used in this project.